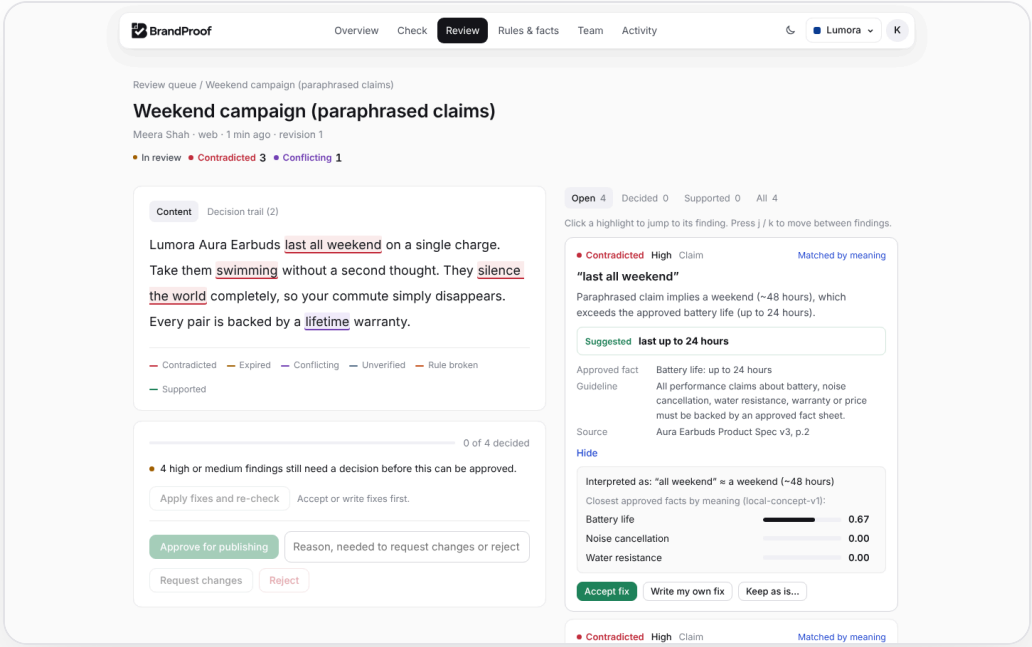
BrandProof

Round 3: The Finish

A pre-publication gate that checks marketing copy and creatives against brand guidelines and approved product facts. Every flag cites its source; a person makes the final call.



Live product	**https://brand-proof-three.vercel.app**
Source code	**https://github.com/nishankkhadpe7-afk/BrandProof**
API docs	**https://brandproof-api-h2pc.onrender.com/docs**
Demo sign-in	One-click demo accounts on the sign-in page (password **demo-pass-2026**)

Team false 9s · SIES Graduate School of Technology · November 2026

Summary

What Round 3 delivers

Round 2 proved the idea runs. Round 3 turns it into a product a stranger can use alone: the three promised improvements are shipped, Round 2's open items are closed, and the core journey holds up under bad input, refreshes and repeated actions.

3 / 3
promised improvements delivered

27
automated API tests, passing on SQLite and PostgreSQL

0.33 → 0.90
recall on paraphrased false claims (40-case set)

365
hand-labelled benchmark sentences (325 real, from Kaggle)

Marking area	Where to look
Completeness	Features list (p.8), full journey (p.15)
Three improvements	Pages 3–6
Reliability	Testing and robustness (pp.12–14)
Usability	Interface (p.7), journey (p.15)
Documentation and roadmap	Architecture (pp.9–10), README, roadmap (p.17)

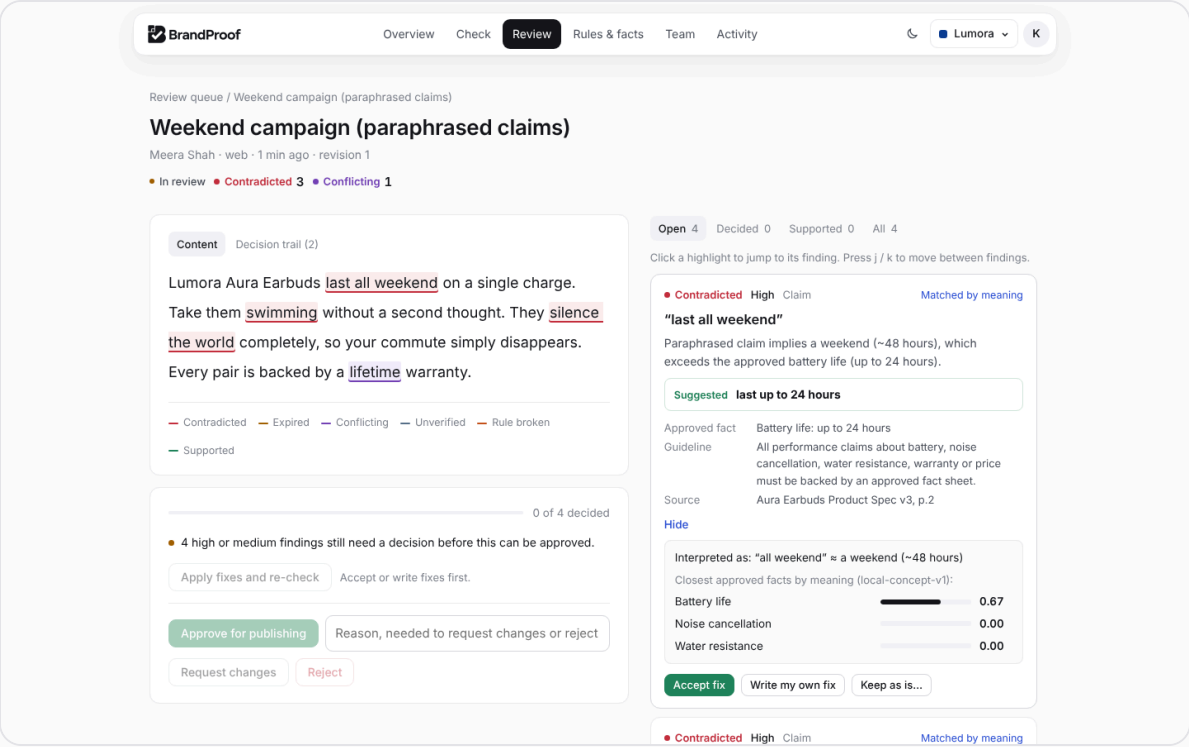
> Everything described here was built during the event and is live at the link on the cover.

The three improvements we committed to, delivered

Promised in Round 2	What we said	What shipped	Status
01 Semantic evidence	Embedding retrieval (pgvector) matches paraphrased claims like “lasts all weekend” to the right fact.	Every fact and claim-like sentence is embedded; the nearest fact is retrieved and the paraphrase is *interpreted* into a comparable value (“all weekend” ≈ 48 h vs “up to 24 h”). Findings show a “Why this fact?” trace. Gemini embeddings are ranked by **pgvector** in PostgreSQL when configured; an offline embedder always runs.	**Done**
02 OCR for creatives	Read text inside banners and scanned PDFs, check it with the same rules, and mark the region.	Tesseract reads every word with its bounding box. Image text and scanned PDF pages go through every text check, and each finding is drawn as a box on the creative, per page. Gemini vision is a fallback; with no engine the result says so. Corrected creatives can be uploaded as a new revision.	**Done**
03 Accounts + workspaces	Email login, real users in roles, and isolated per-brand workspaces with their own audit logs.	Email and password accounts (PBKDF2, hashed session tokens, login throttling, password reset by email), roles per workspace (one person can hold several), email invitations, last-admin protection, and full isolation: rules, facts, content, images and the activity log are scoped to the workspace.	**Done**

> Nothing promised was dropped. One honest caveat: on real-world copy, semantic matching adds coverage for paraphrases but does not beat the exact rules on overall F1 (p.13).

Semantic evidence: paraphrased claims are matched to the right fact



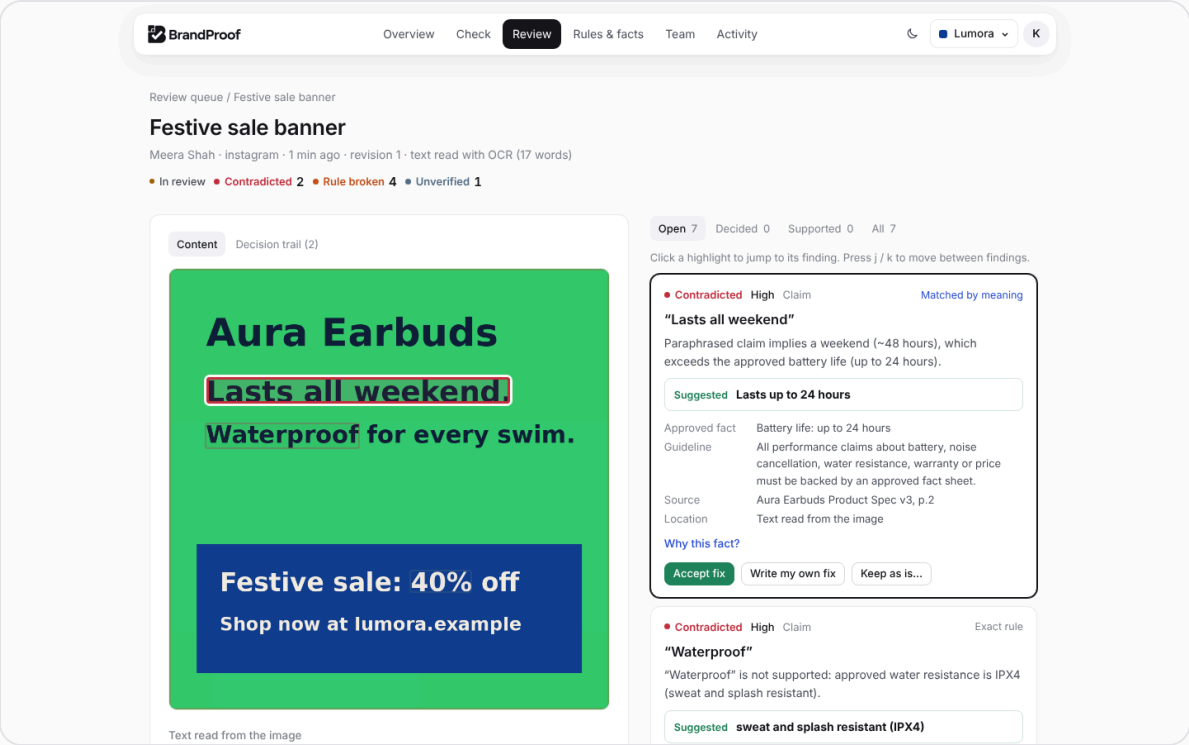
1. **Embed.** Each fact is embedded from its label, aliases, value, approved wording, ruled-out phrases and example paraphrases a brand owner adds.
2. **Retrieve.** The closest fact by cosine similarity. With PostgreSQL, Gemini vectors are stored in a `vector` column and ranked in SQL (`<=>`).
3. **Interpret.** Vague wording becomes a number for the fact's unit: all day ≈ 16 h, all weekend ≈ 48 h, a week ≈ 168 h, “lifetime” or “complete silence” = unbounded.
4. **Decide.** Same comparator, validity and conflict rules as exact claims. If wording can't be interpreted it stays *unverified*, never auto-approved.

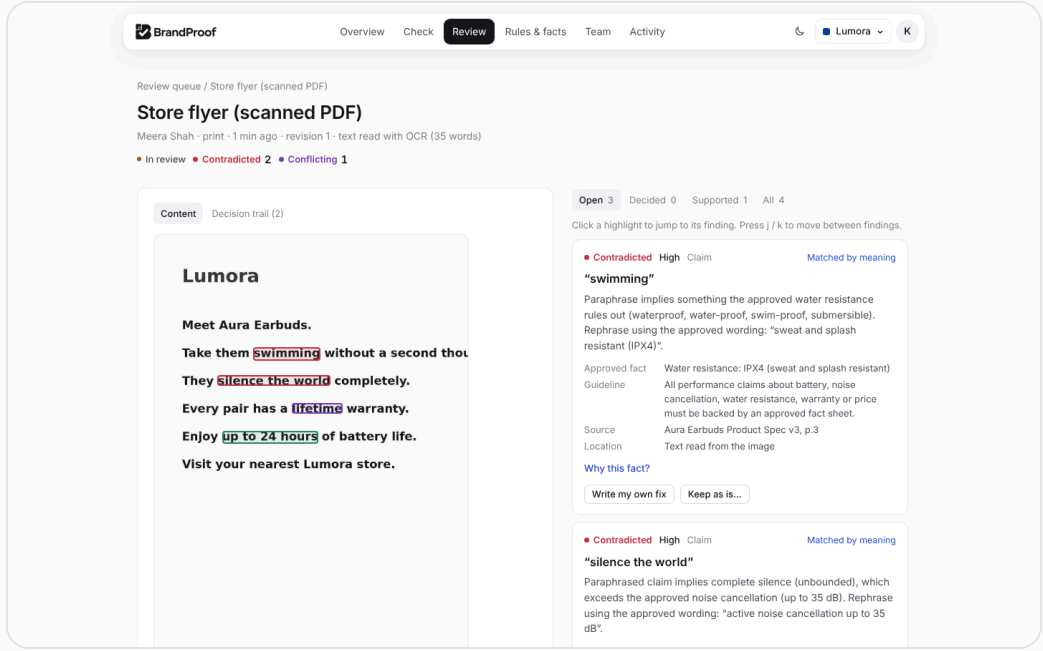
40 paraphrase cases	Precision	Recall	F1
Exact rules only (Round 2)	1.00	0.33	0.50
+ semantic (Round 3)	0.95	**0.90**	**0.93**

A “Test a claim” tool shows which fact a sentence retrieves and how it would be judged.

Improvement 02

OCR: text inside banners and scanned PDFs is checked and boxed

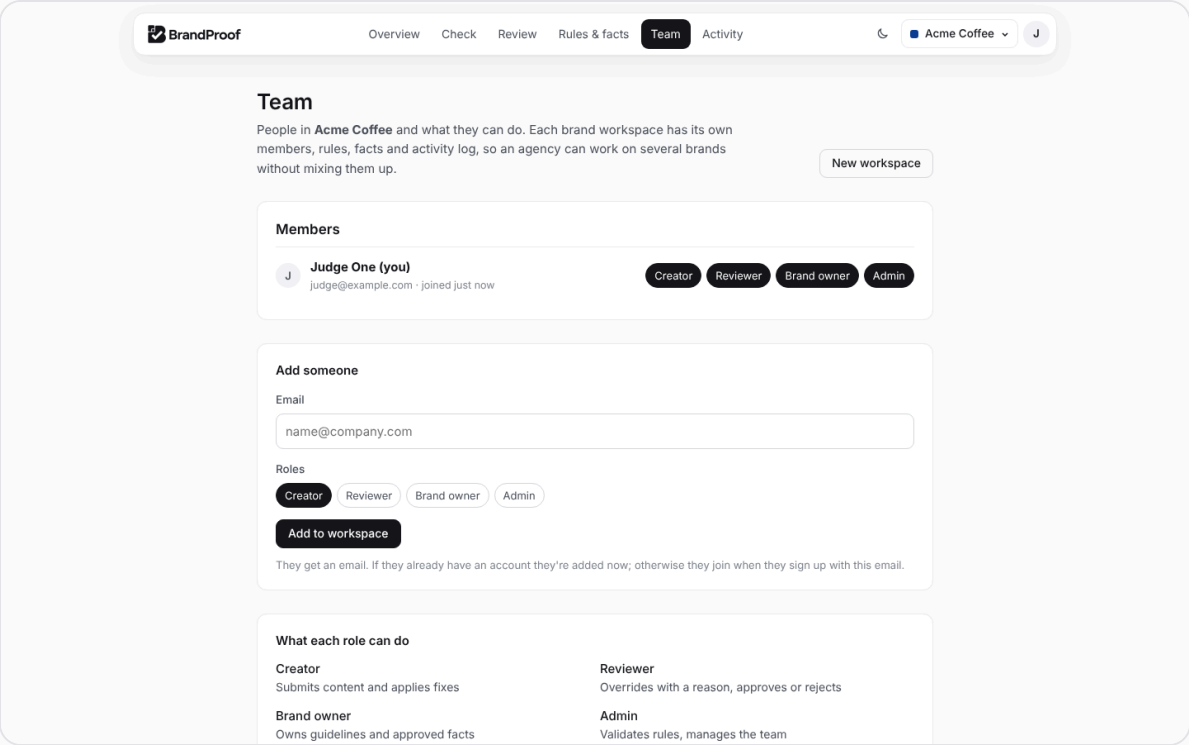




- Two Tesseract modes (layout and sparse text), small images upscaled, words under 55% confidence dropped.
- PDF pages with a text layer use it; pages without one are rendered at 150 dpi and read, up to 10 pages.
- Each finding's text span is mapped back to the union of word boxes and drawn on the right page.
- Works in the Figma plugin too: findings map back to the exact text layer.

Improvement 03

Accounts and brand workspaces

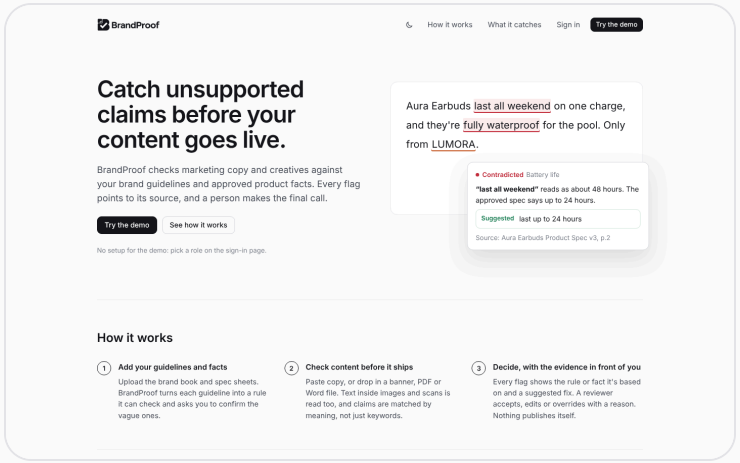


Capability	How
Sign-up / sign-in	Email + password (PBKDF2-SHA256, 240k iterations); 14-day hashed bearer sessions; optional “keep me signed in”
Password reset	Single-use emailed link, 1-hour expiry, signs out other devices, no account enumeration
Roles	Creator, Reviewer, Brand owner, Admin, set per workspace and enforced on every API call
Invitations	Email invite; joins automatically on sign-up with that address
Isolation	Every row carries a workspace id; other workspaces' records return 404; images served only through the authenticated API
Agencies	One person can work across several brands and switch from the top bar

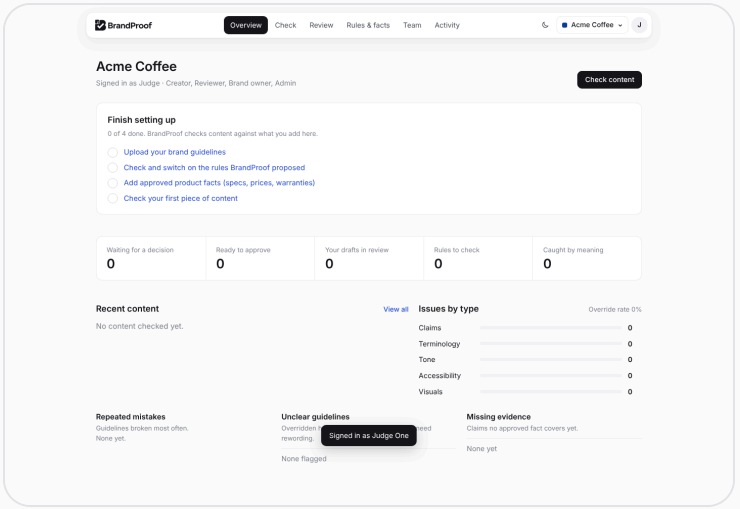
Tested: registration, isolation, cross-workspace access, invites, last-admin protection, password reset (single use, expiry).

Usability

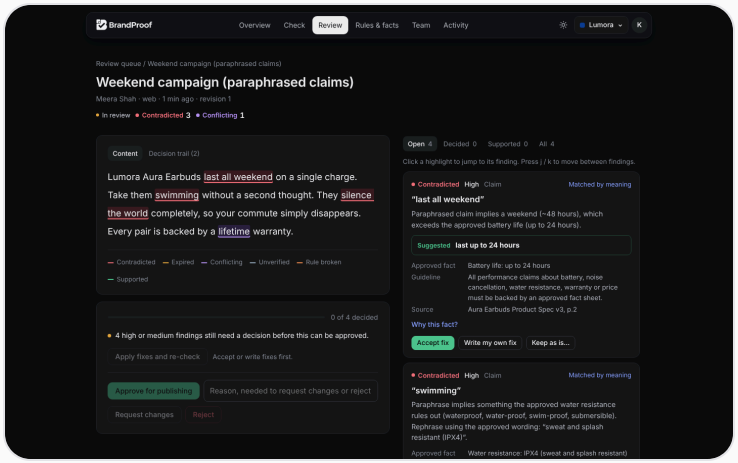
An interface a judge can use without a walkthrough



Landing page explains the product and offers a one-click demo.



A new workspace shows a setup checklist instead of empty screens.



Light and dark themes; the logo animates when switching.

- Role-aware home: what's waiting for you, rules to check, ready to approve.
- Live “Checking…” state while a file is read and checked in the background.
- Plain-language errors (“That file couldn't be opened…”) instead of technical messages.
- Keyboard: j / k between findings; visible focus; reduced motion respected.
- Drag-and-drop upload, example copy, and a “Test a claim” tool.
- Responsive down to phone width.

Complete feature list

Guidelines → rules

- Paste or upload PDF (incl. scans), DOCX, TXT, MD
- 14 typed rule kinds, each linked to source and line
- Vague, untyped, incomplete and conflicting rules stay off until checked
- Edit type and values; switch on / off
- Topics extracted for any domain (“caffeine claims…”)

Evidence ledger

- Facts with value, unit, comparator, aliases, example paraphrases, ruled-out phrases, approved wording, source, owner, validity dates
- Expired and conflicting facts detected
- Edit, remove, test a claim

Checking content

- Text, PDF, DOCX, PNG / JPG / WEBP
- Claims: supported, contradicted, expired, conflicting, unverified
- Exact numbers with unit conversion (days / months / years; mg, ml, kg, kcal, W, GB…)
- Semantic matching of paraphrases
- OCR with boxes on images and scanned pages
- Terms, tone, exclamations, disclaimers, sentence length, all caps, link text, alt text
- Visual palette, contrast and resolution
- Optional AI tone and claim checks (Gemini) with fallback
- Background checks with retries; live status

Review and governance

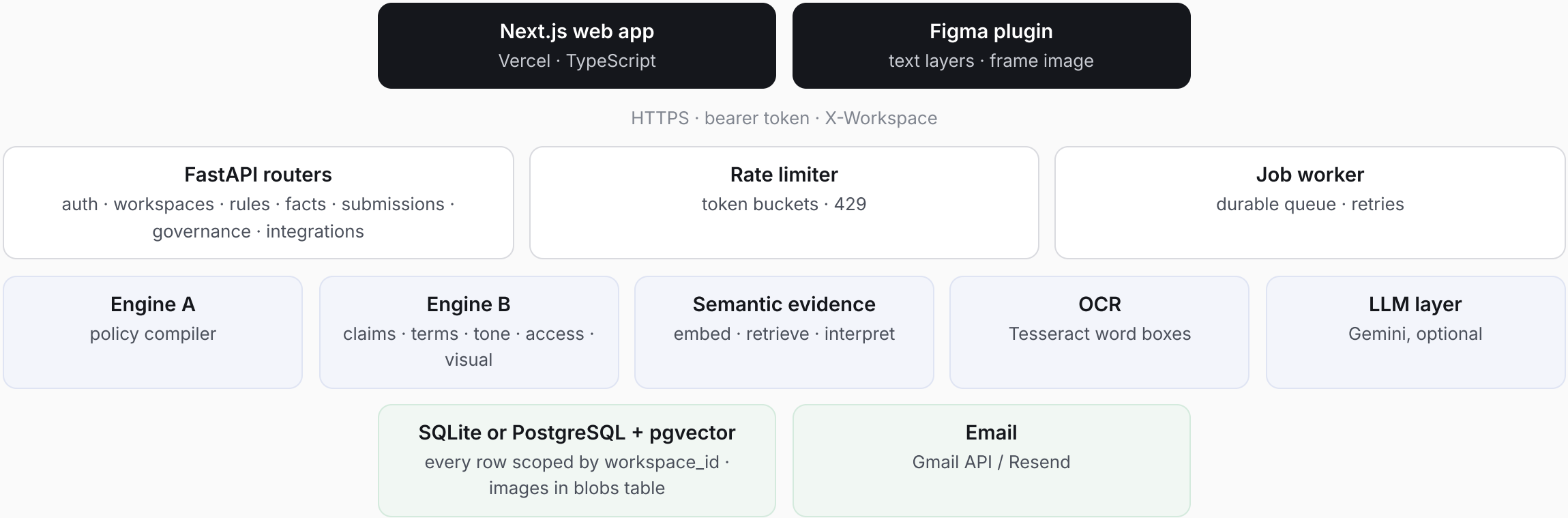
- Accept fix, write own fix, override with reason
- Apply fixes → new revision, re-checked; overrides carry forward
- Upload corrected creative as next revision
- Approval blocked while high / medium findings are open
- Decision trail and append-only activity log
- Insights: repeated mistakes, unclear rules, missing evidence

Accounts and platform

- Accounts, roles, workspaces, invites, password reset (Gmail API or Resend)
- Figma plugin: check selection, show layer, apply fix in place
- Rate limiting, upload validation, SQLite or PostgreSQL + pgvector
- Landing, privacy, terms; light / dark theme

Architecture

Technical architecture



Layer	Technology	Layer	Technology
Frontend	Next.js 15, React 19, TypeScript (Vercel)	Backend	FastAPI, Pydantic, Python 3.12, Docker (Render)
Documents	PyMuPDF, python-docx, Pillow, Tesseract	Data	SQLite (dev) · PostgreSQL + pgvector (prod option)

How the important parts work

Deterministic first, AI second

Exact rules and fact comparisons never depend on an LLM. Semantic and AI findings are tagged with their engine and lower confidence, and missing evidence is “unverified”, never “wrong”.

Background checks

An upload is stored and returns at once with status *queued*; a worker thread runs extraction, OCR and checks. The submission row is the durable job record, so unfinished jobs resume after a restart. Jobs are idempotent (findings replaced, never appended) and retry with exponential backoff (2, 4, 8 s) before showing the error with “Check again”.

Approval gate

Server-side: approval is refused while any high or medium finding is pending; overrides need a reason; decisions lock once approved, rejected or superseded.

Security

- PBKDF2 passwords, only token hashes stored, sessions expire
- Role checks on every write, per workspace
- Rate limits: sign-in 10/min, uploads 20/min, writes 90/min, reads 300/min, plus failed-login cap per email
- Uploads type-checked and opened before acceptance; 15 MB and 50,000-character limits
- Secrets only in environment variables; none in the repository

Code layout

`backend/app/` : `routes/*` , `core.py` (identity, scoping), `compiler.py` , `preflight.py` , `semantic.py` , `ocr.py` , `jobs.py` , `ratelimit.py` , `mailer.py` , `db.py` . `frontend/app/` : one folder per page. `figma-plugin/` .

Every Round 2 gap, closed

Round 2 said	Round 3
SQLite instead of PostgreSQL	PostgreSQL + pgvector supported via `DATABASE_URL`; SQLite kept for local use
Synchronous checks (no queue)	Background job queue with retries, crash recovery and live status
Alias matching; embeddings next	Semantic retrieval and paraphrase interpretation (Improvement 01)
Header role switcher, not authentication	Real accounts, sessions, password reset, roles per workspace (Improvement 03)
OCR deferred	OCR for images and scanned PDFs with region boxes (Improvement 02)
Tenant isolation planned	Workspace isolation on every table, image and log entry
Benchmark proposed, not run	365 hand-labelled sentences run on every change (p.13)
Image revisions needed a new upload	“Upload corrected file” creates the next revision; overrides carry forward

Also added after Round 2

Figma plugin · email invitations · rate limiting · upload validation and friendly errors · landing, privacy and terms pages · redesigned interface with dark mode · unit conversion and negation handling · AI model fallback.

How we tested

27 automated API tests (pytest)

- Accounts: sign-in required, registration, isolation, agency across two brands, invites, last admin, password reset
- Round 2 behaviour: evidence statuses, ambiguity gate, roles, approval gate, revise and re-check
- Semantic: paraphrases matched, approved wording not flagged, no double reporting
- OCR: banner text and regions, scanned PDF, upload and corrected-file revision, no-OCR notice
- Platform: job idempotency and retry, background queue, rate limit 429, upload limits, bad files, Gmail mailer, Figma endpoint, AI path (mocked)

All pass on SQLite and on PostgreSQL + pgvector.

End-to-end journeys (browser, scripted)

1. New visitor → sign up → empty workspace → add guidelines → switch on rules → add fact
2. Check content (double-clicked) → one submission, findings appear → refresh → accept and override → approve
3. Demo roles: creator fixes, reviewer approves, admin manages team
4. Banner, scanned PDF, paraphrase pages; theme switch; phone width

Fixed from testing: mg / kcal units ignored; false flag on approved water wording; corrupt files accepted; raw validation errors; rule topics outside the demo domain; retired AI model.

Benchmark on real and paraphrased copy

Set	Method	Precision	Recall	F1	Fact retrieval
Team-written paraphrases (40)	Exact rules only	1.00	0.33	0.50	0.31
	Exact rules + semantic	0.95	**0.90**	**0.93**	**0.76**
Real Kaggle text (325)	Exact rules only	0.96	0.88	0.92	0.50
	Exact rules + semantic	0.89	0.89	0.89	**0.65**

About the data

325 sentences sampled from public Kaggle data (Amazon India and Flipkart listings and reviews, Amazon phone reviews), covering battery, water, noise and warranty mentions plus hard negatives. Hand-labelled with one written rule: flag if, as Aura Earbuds copy, the sentence exceeds the approved spec, implies more than IPX4, or states any warranty term.

What it shows, honestly

Real copy mostly states figures outright, and exact rules catch those. Semantic matching adds paraphrase coverage and better fact retrieval, at the cost of a few false alarms on idioms (“total silence” in a room). Fixes made after seeing results mean this is a development set; a held-out set is on the roadmap. Run it: `python -m eval.semantic_eval --csv eval/kaggle_benchmark.csv`.

Reliability

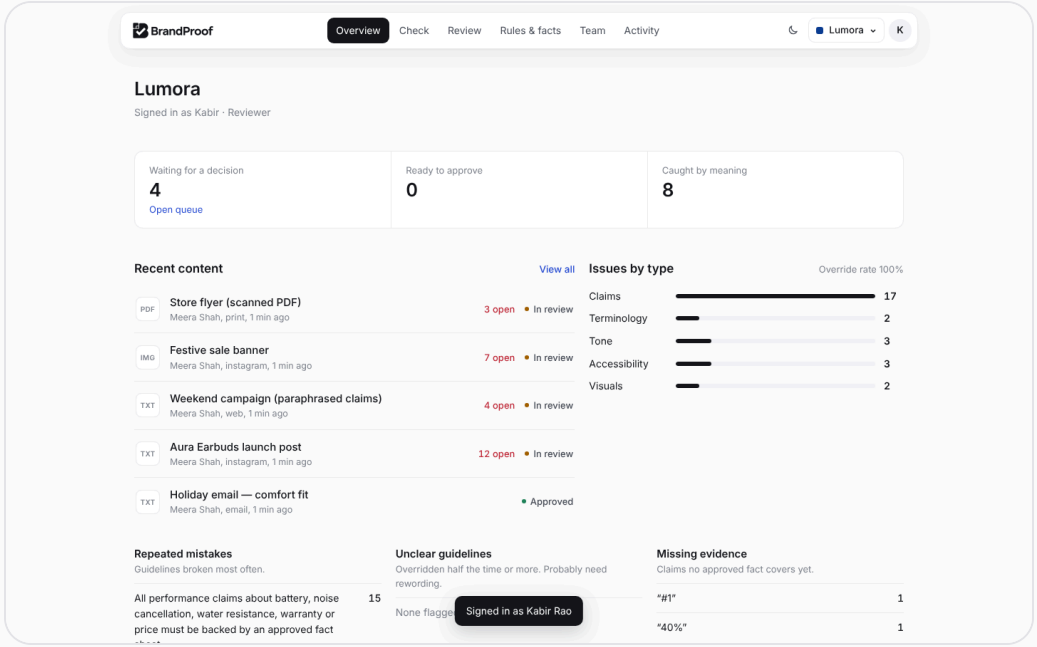
What happens when a judge tries to break it

Action	Result	Action	Result
Empty copy	“Nothing to check…”	Approve twice	Second refused: already approved
Unsupported file (.exe)	Rejected with the allowed types	Decide after approval	Refused: decisions are locked
Corrupt image or PDF, empty file	“That file couldn't be opened / is empty”	Revise a superseded revision	Refused, link to the newer revision
File over 15 MB / text over 50k chars	413 with the limit	Double-click Run check	One submission created
Missing field / malformed request	One readable sentence (“Title is required.”)	Refresh mid-check	Status resumes; page polls until done
Unknown or other-brand item	404 “Not found in this workspace”	Server restart mid-check	Job resumes on start
Expired or fake session	Sign-in page	Repeated wrong passwords	429 with retry time
Activate an incomplete rule	Refused, names the missing value	AI provider down / model retired	Falls back to next model, then to exact checks

> Zero server errors across the scripted bad-input run. Empty states give the next step instead of blank tables.

The main journey, for someone who has never seen it

1. Open the live link and click **Try the demo**.
2. Pick **Meera Shah (Creator)**. Open **Weekend campaign**: claims caught by meaning, with *Why this fact?*
3. Open **Festive sale banner**: claims read from the image and boxed on it.
4. On **Aura Earbuds launch post**, click **Accept fix** on a few findings, then **Apply fixes and re-check**.
5. Sign out and pick **Kabir Rao (Reviewer)**. **Approve** stays disabled until every important finding has a decision; override one with a reason and approve.
6. Check **Activity** for the full record. Switch brand to see **Verdant Oat Co.** (as Meera or Ravi).
7. Or **Create account** for an empty workspace and follow the setup checklist.



What changed from our original plan, and why

Plan	What we did	Why
pgvector as the only vector store	pgvector in PostgreSQL, plus an in-process index on SQLite	Lets the project clone-and-run with no database server; production uses pgvector
Embeddings only	Offline embedder always on; Gemini embeddings added when a key is set	Works without network or cost; deterministic and testable
150-case labelled benchmark	365 cases (325 real Kaggle + 40 paraphrases)	Bigger and grounded in real copy; not yet held-out
Redis / Celery queue	Durable in-process queue on the submission row	One service on Render free tier; same job function moves to Redis later
Email via SMTP	Gmail API over HTTPS (or Resend)	Render's free tier blocks SMTP ports

> None of the three improvements was impossible, so none was dropped or replaced.

Product roadmap and future development

Next 3 months

- Held-out benchmark (150+ new cases) and published results
- Pilots with 2–3 D2C brands and one agency; measure review time and claims caught
- Fact extraction straight from spec-sheet PDFs
- Google Docs and Canva add-ons; publish the Figma plugin

3–6 months

- Industry rule packs: ASCI, FSSAI health claims, SEBI ads, green claims
- Hindi, Tamil and Hinglish copy
- Video and audio: transcripts and on-screen text
- Publishing gate API and webhooks for CMS and schedulers

6–12 months

- Learning loop: overrides suggest rule rewording and new examples
- Cross-encoder reranker and fine-tuned verifier
- SSO, SOC 2 groundwork, regional data hosting
- Multi-instance queue (Redis / Postgres) and horizontal scaling

Business model	Free solo · Team ₹3–5k per brand per month · Agency per seat · Enterprise (SSO, rule packs, API)
First customers	Indian D2C brands (beauty, food, electronics) and the agencies that serve them; wedge: a free claim audit of the last 50 posts

Team and contributions

Participant (full legal name)	Role	Contribution
Nishank Khadpe	Product and research	User needs, scope, evaluation
Namish Sharma	Policy compiler and NLP	Rule extraction, ambiguity
Vansh Jain	Evidence ledger and backend	Retrieval, APIs, security
Samar Gharat	Frontend and UX	Workflow, testing, accessibility

Run it yourself

Windows: double-click `start.bat`. Otherwise: `pip install -r backend/requirements.txt`, `uvicorn app.main:app`, then `npm install && npm run dev` in `frontend`. Full steps, configuration and deployment in the README.

Links

Live: https://brand-proof-three.vercel.app
Code: https://github.com/nishankkhadpe7-afk/BrandProof
API: https://brandproof-api-h2pc.onrender.com/docs